Eventify Platform

High-Level System Design Document

Phase 1 Submission

SWAPD 351 – Software Architecture and Design

Spring 2026

Team Members

Habiba Magdy
Basmala Salah
Arwa Mohamed
Lana Mohamed

Zewail City of Science and Technology

University of Science and Technology

Document Version 1.0 — March 28, 2026

Contents

Requirements Specification Document

Project Overview

Eventify is an event management platform built on a microservices architecture. The system allows users to create, browse, and RSVP to events, communicate through real-time chat, and view event analytics on a live dashboard. The platform uses OAuth2 (Google) for authentication, an API Gateway for request routing, and asynchronous messaging for inter-service communication.

Functional Requirements

FR-1 **User Registration and Authentication**: Users shall be able to register and log in using Google OAuth2. The system shall issue JWT tokens upon successful authentication.

FR-2 **Event Creation**: Authenticated users shall be able to create events with a title, description, date, time, location, and maximum capacity.

FR-3 **Event Browsing**: Users shall be able to browse and search for events by date, category, or keyword.

FR-4 **Event RSVP**: Authenticated users shall be able to RSVP to events. The system shall enforce capacity limits.

FR-5 **Event Update and Deletion**: Event creators shall be able to update or delete their own events.

FR-6 **Real-Time Chat**: Users who have RSVP'd to an event shall be able to send and receive messages in an event-specific chat room.

FR-7 **Notifications**: The system shall send notifications when a user RSVPs, when an event is updated, or when a new chat message is posted.

FR-8 **Dashboard**: An analytics dashboard shall display real-time data including total events, total RSVPs, trending events, and user activity metrics.

FR-9 **User Profile**: Users shall be able to view and edit their profile information and see their RSVP history.

Non-Functional Requirements (Quality Attributes)

NFR-1 **Scalability**: The system shall scale horizontally. Each microservice shall be independently deployable and scalable. The database shall support sharding for high-volume data.

NFR-2 **Performance**: API response times shall be under 200ms for read operations under normal load. The dashboard shall refresh data within 2 seconds.

NFR-3 **Availability**: The system shall target 99.9% uptime. Circuit breakers shall prevent cascading failures across services.

NFR-4 **Security**: All API endpoints shall require valid JWT tokens. Data in transit shall be encrypted using TLS. OAuth2 tokens shall be short-lived with refresh capability.

NFR-5 **Resilience**: The system shall implement circuit breakers on all inter-service calls and retry logic with exponential backoff for transient failures.

NFR-6 **Observability**: Centralized logging (OpenSearch), application metrics (Prometheus + Grafana), and distributed tracing shall be implemented.

NFR-7 **Maintainability**: Each service shall follow clean architecture principles. Services shall be containerized using Docker and orchestrated using Docker Compose.

User Stories / Use Cases

ID	Title	Description
US-01	User Login	As a user, I want to log in with my Google account so that I do not need to create a separate password.
US-02	Create Event	As an authenticated user, I want to create an event with details so that others can discover and join it.
US-03	Browse Events	As a user, I want to search and filter events by date or category so that I can find relevant events quickly.
US-04	RSVP to Event	As an authenticated user, I want to RSVP to an event so that I can reserve my spot.
US-05	Cancel RSVP	As a user, I want to cancel my RSVP if my plans change.
US-06	Event Chat	As an event attendee, I want to chat with other attendees in real-time so that we can coordinate.
US-07	Receive Notifications	As a user, I want to receive notifications about event updates and new messages.
US-08	View Dashboard	As a user, I want to see a dashboard with trending events and activity metrics.
US-09	Edit Profile	As a user, I want to update my profile information and view my event history.
US-10	Delete Event	As an event creator, I want to delete my event if it is cancelled.

Architectural Drivers

The following quality attributes drive the architectural decisions:

- **Scalability**: Independent scaling of services under varying load (e.g., event browsing may spike during weekends). This drives the choice of microservices over a

monolithic architecture.

- **Resilience**: The system must tolerate individual service failures without full system downtime. This drives the adoption of circuit breakers, retries, and asynchronous messaging.
- **Security**: Sensitive user data and authentication tokens must be protected. This drives the choice of OAuth2 with JWT and TLS encryption.
- **Performance**: Read-heavy workloads on event browsing and dashboard require caching. This drives the adoption of Redis for caching and session management.
- **Observability**: In a distributed system, tracing requests across services is critical for debugging. This drives centralized logging with OpenSearch and monitoring with Prometheus/Grafana.

C4 Model Diagrams

This section presents the system architecture using the C4 model at three levels of abstraction. Full-resolution diagrams are provided alongside this document in PNG/SVG format.

Level 1: Context Diagram

The Context Diagram shows Eventify as a single system, its users, and external dependencies.

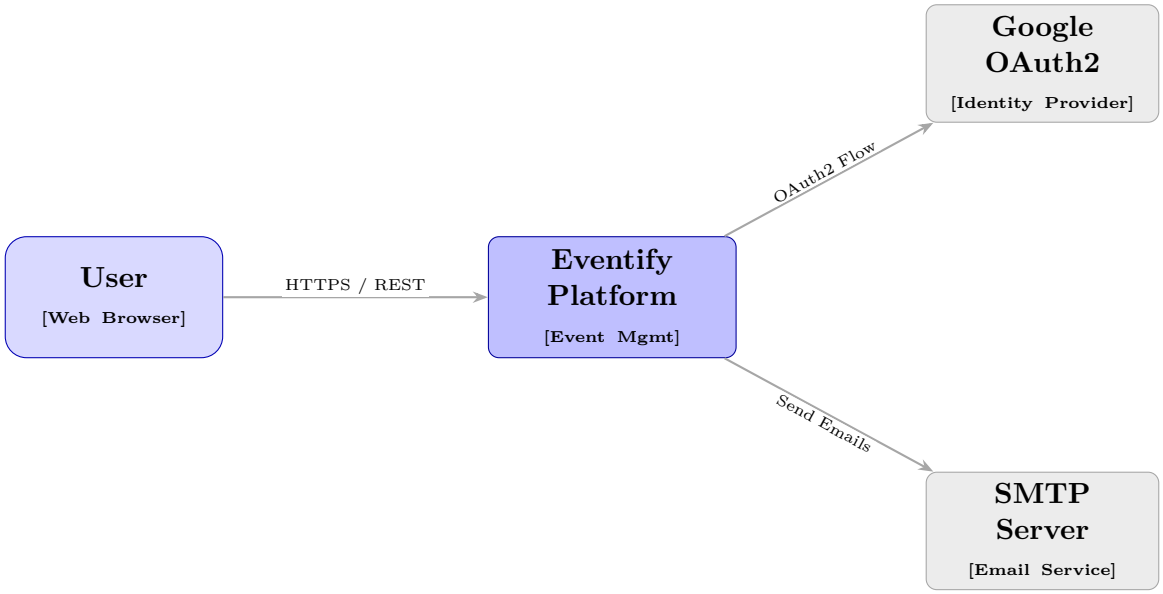

Figure 1: Level 1 – System Context Diagram

Description: Users interact with the Eventify Platform through a web browser over HTTPS. The system depends on Google OAuth2 for authentication and an SMTP server for sending email notifications.

Level 2: Container Diagram

The Container Diagram breaks Eventify into its constituent containers (services, databases, caches).

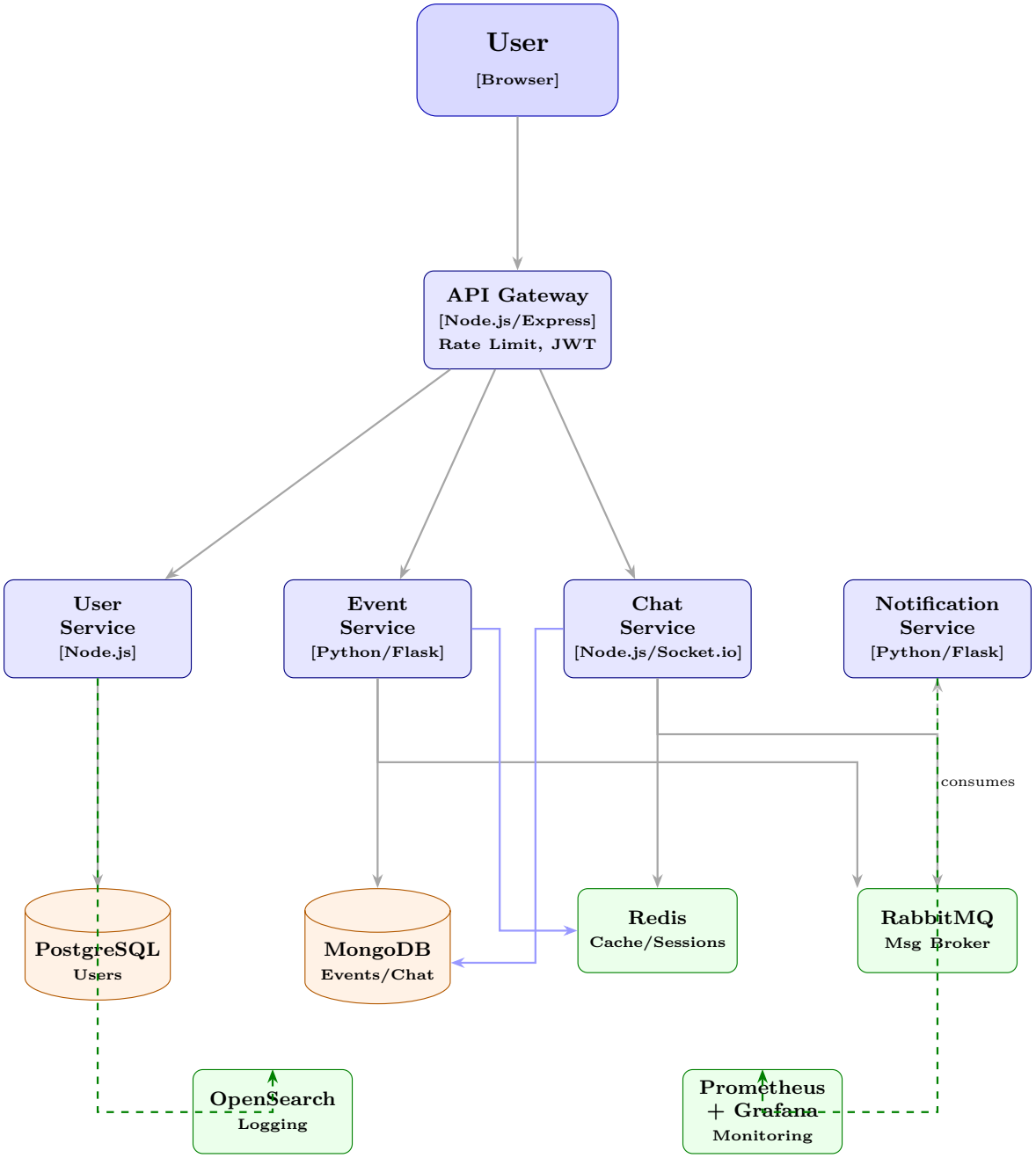

Figure 2: Level 2 – Container Diagram

Container Descriptions:

- **API Gateway (Node.js/Express)**: Entry point for all client requests. Routes requests to appropriate services, handles rate limiting, and validates JWT tokens.
- **User Service (Node.js/Express)**: Manages user registration, OAuth2 authentication with Google, profile management, and JWT token issuance.
- **Event Service (Python/Flask)**: Handles CRUD operations for events and RSVPs. Stores data in MongoDB. Publishes events to RabbitMQ on state changes.
- **Notification Service (Python/Flask)**: Consumes messages from RabbitMQ and sends notifications via WebSocket push and email (SMTP).

- **Chat Service (Node.js/Socket.io)**: Provides real-time messaging using WebSockets. Stores chat history in MongoDB.
- **PostgreSQL**: Relational database for user data and authentication records. Supports sharding through Citus extension.
- **MongoDB**: Document database for events, RSVPs, and chat messages. Supports horizontal scaling via sharding.
- **Redis**: In-memory store for session management, caching frequently accessed event data, and rate limiting counters.
- **RabbitMQ**: Message broker for asynchronous communication between services (event updates, notifications).
- **OpenSearch**: Centralized log aggregation and search for all services.
- **Prometheus + Grafana**: Metrics collection and visualization for monitoring application and host metrics.

Level 3: Component Diagram – Event Service

The Component Diagram shows the internal structure of the Event Service.

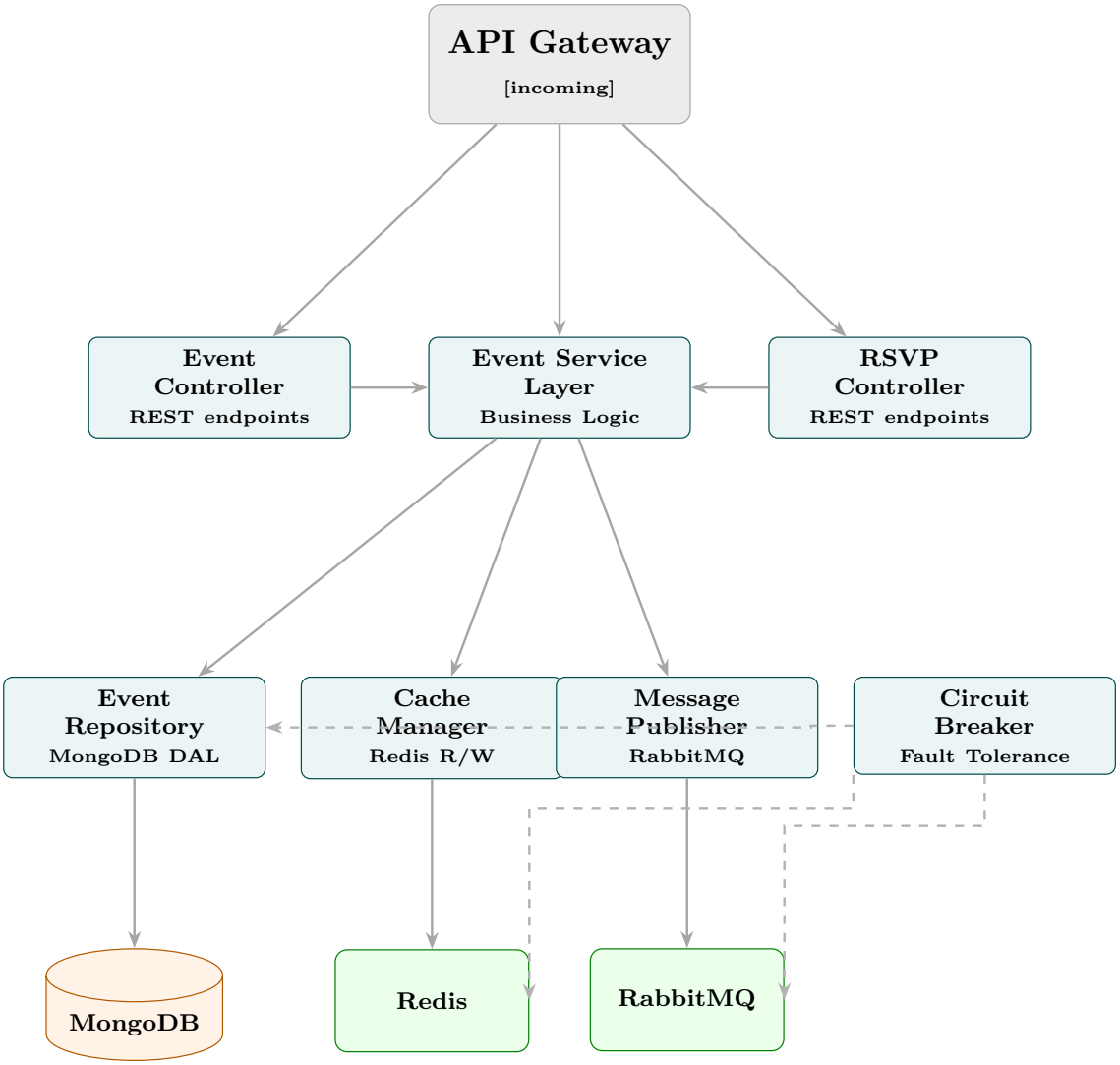

Figure 3: Level 3 – Component Diagram of Event Service

Interaction Flow:

1. A request arrives at the Event Controller via the API Gateway.
2. The Controller delegates to the Event Service Layer for business logic.
3. The Service Layer checks Redis cache (via Cache Manager) before querying MongoDB (via Event Repository).
4. On write operations, the Message Publisher sends an event to RabbitMQ to notify other services.
5. All external calls (Redis, RabbitMQ, MongoDB) are wrapped with Circuit Breakers.

Architecture Decision Records (ADRs)

ADR-001: Adoption of Microservices Architecture

Status	Accepted
Date	2026-03-15
Decision	Adopt a microservices architecture over a monolithic architecture.

Context: The Eventify platform comprises distinct domains: user management, event management, real-time chat, and notifications. These domains have different scaling characteristics. Event browsing may experience high read traffic, while chat requires persistent WebSocket connections. A monolithic application would force all domains to scale together, wasting resources.

Decision: Decompose the system into five independently deployable microservices: API Gateway, User Service, Event Service, Chat Service, and Notification Service. Each service owns its data and communicates via REST (synchronous) or RabbitMQ (asynchronous).

Consequences:

- *Positive*: Independent scaling, independent deployment, technology flexibility per service, fault isolation.
- *Negative*: Increased operational complexity, network latency between services, eventual consistency challenges, need for distributed tracing.

Alternatives Considered:

- Monolithic architecture — simpler to develop initially but does not meet scalability and independent deployment requirements.
- Service-oriented architecture (SOA) — heavier governance and shared data layer; microservices provide better autonomy.

ADR-002: Database Selection – PostgreSQL and MongoDB

Status	Accepted
Date	2026-03-15
Decision	Use PostgreSQL for user data and MongoDB for event/chat data.

Context: The system handles two fundamentally different data patterns. User data (accounts, credentials, OAuth tokens) is relational, requires strong consistency, and benefits from ACID transactions. Event data (events with varying attributes, RSVPs, chat messages) is document-oriented, with flexible schemas and high write throughput requirements.

Decision: Adopt a polyglot persistence strategy:

- **PostgreSQL** for the User Service: stores user profiles, OAuth tokens, and session metadata. Supports sharding via the Citus extension for horizontal scaling.

- **MongoDB** for the Event Service and Chat Service: stores events, RSVPs, and chat messages. MongoDB's document model naturally maps to event objects with nested attendee lists. Native sharding support enables horizontal scaling by event ID or date range.

Consequences:

- *Positive*: Each database is optimized for its workload. PostgreSQL guarantees consistency for user operations. MongoDB handles high-throughput writes for chat and flexible event schemas.
- *Negative*: Operational overhead of managing two database technologies. Cross-database joins are not possible; data must be denormalized or fetched via API calls.

Sharding Considerations:

- PostgreSQL: Shard by user ID using Citus distributed tables.
- MongoDB: Shard events collection by `event_id` (hashed) for even distribution. Shard chat messages by `event_id` to co-locate messages for the same event.

ADR-003: OAuth2 via Google with JWT Tokens

Status	Accepted
Date	2026-03-15
Decision	Use Google OAuth2 for user authentication and JWT tokens for session management.

Context: The project requires token-based authentication with OAuth2 via Google as the primary provider. In a microservices architecture, session state must be shareable across services without a centralized session store bottleneck.

Decision: Implement the OAuth2 Authorization Code flow with Google as the identity provider. Upon successful authentication, the User Service issues a signed JWT containing the user's ID and roles. Subsequent requests include this JWT in the Authorization header. The API Gateway validates the JWT signature before routing requests to backend services.

Token Strategy:

- Access Token: JWT with 15-minute expiry, signed with RS256.
- Refresh Token: Opaque token stored in Redis with 7-day expiry.
- Token refresh is handled by the User Service via the `/api/auth/refresh` endpoint.

Consequences:

- *Positive*: Stateless authentication at the API Gateway (no database lookup per request). Users benefit from Google's security infrastructure. JWTs are self-contained and can be verified by any service.
- *Negative*: JWT revocation is not immediate (must wait for expiry or maintain a blacklist in Redis). Dependency on Google's OAuth2 service availability.

ADR-004: RabbitMQ as Message Broker

Status	Accepted
Date	2026-03-15
Decision	Use RabbitMQ for asynchronous inter-service communication.

Context: The Event Service needs to notify the Notification Service when events are created, updated, or when users RSVP. Similarly, the Chat Service triggers notification alerts. These operations are side-effects and should not block the primary request-response cycle.

Decision: Adopt RabbitMQ with durable queues and topic exchanges. Producers publish messages to exchanges, and consumers bind queues to receive specific message types.

Queue Design:

- Exchange: `eventify.events` (topic exchange)
- Routing keys: `event.created`, `event.updated`, `event.deleted`, `rsvp.created`, `rsvp.cancelled`, `chat.message`
- Consumer queues: `notification-queue` (binds to all routing keys)

Consequences:

- *Positive*: Decouples producers from consumers. Durable queues ensure message delivery even if a consumer is temporarily down. Topic-based routing allows flexible message filtering.
- *Negative*: Adds infrastructure complexity. Message ordering is guaranteed only within a single queue. Requires monitoring for queue depth and consumer lag.

Alternatives Considered:

- Apache Kafka — better for high-throughput event streaming, but overkill for our notification use case. RabbitMQ's simpler routing model is sufficient.
- Redis Pub/Sub — does not persist messages if the subscriber is offline. Not suitable for reliable notification delivery.

ADR-005: Redis for Caching and Session Management

Status	Accepted
Date	2026-03-15
Decision	Use Redis as the in-memory data store for caching and session management.

Context: The system requires fast access to frequently read data (event listings, user sessions) and a shared session store accessible by multiple service instances. Read-heavy operations like event browsing benefit from caching to reduce database load and improve response times.

Decision: Deploy a Redis instance shared across services for:

- **Caching**: Cache popular event listings and event details with a TTL of 5 minutes. Cache invalidation occurs on event updates via a write-through strategy.
- **Session Management**: Store refresh tokens and active session metadata. Sessions expire after 7 days of inactivity.
- **Rate Limiting**: Store request counters per user with a sliding window of 1 minute.

Consequences:

- *Positive*: Sub-millisecond read latency. Reduces load on PostgreSQL and MongoDB. Atomic operations for rate limiting counters.
- *Negative*: Additional infrastructure component. Data loss on restart if persistence is not configured (mitigated by using Redis with AOF persistence). Cache invalidation must be carefully managed to avoid stale data.

ADR-006: Multi-Language Service Implementation

Status	Accepted
Date	2026-03-15
Decision	Implement services in both Node.js and Python based on domain requirements.

Context: The project description states that not all microservices need to be implemented in the same programming language, and the language choices should be justified in the ADR. Different services have different runtime characteristics that favor different languages.

Decision:

- **Node.js (JavaScript)**: Used for the API Gateway, User Service, and Chat Service. Node.js excels at I/O-bound operations and has native support for WebSockets via Socket.io. Passport.js provides a mature OAuth2 integration for the User Service. The event-driven, non-blocking architecture is ideal for handling many concurrent WebSocket connections in the Chat Service.
- **Python (Flask)**: Used for the Event Service and Notification Service. Python offers concise syntax for CRUD operations and data validation with libraries like Marshmallow. The team has strong familiarity with Python. Flask's simplicity allows rapid development of RESTful APIs without unnecessary abstraction.

Consequences:

- *Positive*: Each service uses the language best suited to its workload. Team members can work in their strongest language. Demonstrates polyglot microservices as required.
- *Negative*: Requires maintaining build tooling and dependency management for two ecosystems (npm and pip). Shared libraries or utilities cannot be easily reused across languages.

API Specifications

The full OpenAPI 3.0 specifications for both APIs are provided in the `api-specs/` directory. Below is a summary of the endpoints.

Synchronous vs. Asynchronous Communication

Synchronous (REST over HTTP):

- Client → API Gateway → User Service (login, profile)
- Client → API Gateway → Event Service (CRUD, RSVP)
- These are request-response interactions where the client expects an immediate response.

Asynchronous (RabbitMQ Message Broker):

- Event Service → RabbitMQ → Notification Service (event updates, RSVP confirmations)
- Chat Service → RabbitMQ → Notification Service (new message alerts)
- These are fire-and-forget interactions. The producer does not wait for the consumer to process the message.

Justification: Synchronous communication is used for user-facing operations that require immediate feedback (e.g., creating an event and receiving a confirmation). Asynchronous communication is used for side-effects like notifications, where the user does not need to wait for the notification to be delivered before receiving a response to their original request. This decouples services and improves resilience — if the Notification Service is down, event creation still succeeds, and notifications are delivered once the service recovers (messages are persisted in RabbitMQ).

Event Service API – Summary

Method	Endpoint	Description
GET	`/api/events`	List all events (pagination, filtering)
POST	`/api/events`	Create a new event (requires auth)
GET	`/api/events/{id}`	Get event details by ID
PUT	`/api/events/{id}`	Update an event (creator only)
DELETE	`/api/events/{id}`	Delete an event (creator only)
POST	`/api/events/{id}/rsvp`	RSVP to an event (requires auth)
DELETE	`/api/events/{id}/rsvp`	Cancel RSVP (requires auth)
GET	`/api/events/{id}/attendees`	List attendees of an event

User Service API – Summary

Method	Endpoint	Description
GET	`/api/auth/google`	Initiates Google OAuth2 login flow
GET	`/api/auth/google/callback`	OAuth2 callback, issues JWT
POST	`/api/auth/refresh`	Refresh an expired JWT token
GET	`/api/users/me`	Get current user profile
PUT	`/api/users/me`	Update current user profile
GET	`/api/users/{id}`	Get user profile by ID

Resilience Strategies

Circuit Breaker Pattern

All inter-service HTTP calls are protected by circuit breakers. When a downstream service (e.g., the Notification Service) fails consecutively, the circuit opens and immediately returns a fallback response instead of waiting for a timeout.

Implementation: The circuit breaker has three states:

- **Closed**: Requests pass through normally. Failures are counted.
- **Open**: After a threshold of failures (e.g., 5 failures in 30 seconds), the circuit opens. All requests immediately fail with a fallback response.
- **Half-Open**: After a cooldown period (e.g., 10 seconds), a single test request is allowed through. If it succeeds, the circuit closes. If it fails, the circuit remains open.

Libraries:

- Node.js services: `opossum` (circuit breaker library for Node.js)
- Python services: `pybreaker` (circuit breaker library for Python)

Retry with Exponential Backoff

Transient failures (network timeouts, temporary service unavailability) are handled with automatic retries. Each retry waits exponentially longer: 1s, 2s, 4s, up to a maximum of 3 retries.

Justification: Transient failures are common in distributed systems. Retrying with backoff avoids overwhelming a recovering service while still recovering from temporary issues.

Asynchronous Messaging for Decoupling

RabbitMQ provides durable message queues between services. If the Notification Service is temporarily unavailable, messages are persisted in RabbitMQ and delivered when the service recovers. This prevents data loss and decouples service availability.

Health Checks

Each service exposes a `/health` endpoint that returns the service's health status, including connectivity to its databases and caches. The API Gateway periodically checks these endpoints and removes unhealthy services from routing.

Rate Limiting

The API Gateway implements rate limiting (100 requests per minute per user) to prevent abuse and protect backend services from being overwhelmed.

Justification Summary

Mechanism	Justification
Circuit Breaker	Prevents cascading failures when a downstream service is unresponsive. Essential in microservices where one slow service can block all requests.
Retry with Backoff	Recovers from transient network errors without overwhelming the target service. The exponential backoff prevents retry storms.
Message Queue (RabbitMQ)	Decouples producers from consumers. Ensures notifications are eventually delivered even if the consumer is temporarily down.
Health Checks	Enables the API Gateway to route traffic only to healthy instances, improving overall system availability.
Rate Limiting	Protects against denial-of-service attacks and ensures fair resource allocation among users.

Technology Stack Summary

Component	Technology	Rationale
API Gateway	Node.js/Express	Lightweight, high-throughput, large ecosystem for middleware
User Service	Node.js/Express	Passport.js provides mature OAuth2 integration
Event Service	Python/Flask	Team familiarity with Python; Flask is minimal and flexible
Chat Service	Node.js/Socket.io	Native WebSocket support for real-time messaging
Notification Service	Python/Flask	Consumes RabbitMQ messages; Python has good AMQP libraries
User Database	PostgreSQL	ACID compliance for user and auth data; supports sharding via Citus
Event Database	MongoDB	Flexible schema for events; native sharding support
Cache / Sessions	Redis	Sub-millisecond reads; widely used for caching and sessions
Message Broker	RabbitMQ	Reliable message delivery; supports durable queues and routing
Logging	OpenSearch	Full-text search on logs; Kibana-compatible dashboards
Monitoring	Prometheus + Grafana	Industry standard for metrics collection and visualization
Containerization	Docker + Compose	Consistent environments; easy local development and deployment